ALGORITHMS AND COMPLEXITY

1. a) Give a tight bound for the running time of each of the following algorithms as a function of the variable n, and justify your result. You may assume basic arithmetic operations and memory allocation are $O(1)$.

i) Compute the outer product $\mathbf{x}\mathbf{y}^\mathsf{T}$ of two column vectors $\mathbf{x}$, $\mathbf{y}$ of length n.

```
function OuterProduct(x, y, n)
    Allocate memory for an n × n matrix C.
    for i = 0 to n − 1 do
        for j = 0 to n − 1 do
            C_ij = x_i y_j
        end for
    end for
    return C
end function
```

[5]

Answer:
Initialising the matrix is $O(1)$ because memory allocation is $O(1)$ and we do not need to fill it with an initial value. There are two nested loops over n items each, so the inner code is executed n^2 times. The inner code is $O(1)$. Therefore the algorithm is $O(n^2)$.
Almost everyone answered this correctly. A few gave $O(n^3)$ for matrix multiplication.

ii) Given an item x and a sorted array A of length n, compute whether or not the item x is in A using bisection:

```
function ElementInSortedArray(x, A)
    L = 0
    R = len(A) − 1
    while L ≤ R do
        m = floor((L + R)/2)
        if A_m = x then
            return True
        else if A_m < x then
            L = m + 1
        else
            R = m − 1
        end if
    end while
    return False
end function
```

[5]

Answer:
The basic idea is to note that at each step of the algorithm, the size of the array still to be searched halves, and therefore after $\log_2$(size of array) the algorithm must stop, i.e. after $\log_2 n = O(\log n)$. Fully precise ar-

Answers

gument follows:

The number of items in A that are candidates to be equal to x at a given iteration is $R-L+1$, which is n at the beginning of the algorithm. After the first iteration, the algorithm either returns True, or sets $L=m+1$ or $R=m-1$. Suppose $L=m+1$, then $m=\text{floor}((n-1)/2)=(n-1)/2-\theta$ where $\theta=0$ or $\theta=1/2$. In this case, at the beginning of the next iteration, $R-L+1=(n-1)-((n-1)/2-\theta+1)+1=(n-1)/2+\theta\leq n/2$. Similarly, if $R=m-1$ then $R-L+1=(n-1)/2-\theta-1+1=(n-1)/2-\theta\leq n/2$. So, in every case, after the first iteration, there are $\leq n/2$ possible candidate elements of A that could be equal to x. So after $k=\text{ceil}(\log_2 n)+1$ iterations, there will be <1 candidates, so the algorithm must have stopped. So $\log_2 n+2$ is an upper bound for the number of iterations, and since each statement is $O(1)$ the total cost is $O(\log n)$.

An alternative approach is to note that the algorithm is equivalent to a recursive one where the while loop is remained, and at the end it is called on the subarray of items from L to R where the size of that subarray is $\leq n/2$. So $T(n)=T(n/2)+O(1)$ and therefore (applying the master theorem with $a=1,b=2,d=0$) $T(n)=O(\log n)$.
Quite a few people misunderstood what the algorithm was doing and gave $O(n)$.

b) Give a tight bound for each of the following recurrence relations. Note that the Master Theorem is stated in full on the final page.

i) $T(n)=4T(n/5)+O(\sqrt{n})$. [2]

Answer:
$a=4,b=5,d=0.5$ so $\log_b a=\log_5 4\approx 0.86>d$ so:
$T(n)=O(n^{\log_b a})\approx O(n^{0.86})$.
Most gave the correct answer. A few accidentally computed $\log_4 5$.

ii) $T(n)=\max\{1,nT(n-1)\}$. [2]

Answer:
The master theorem doesn't apply here. Assume the definition holds for $n=0$ (i.e. $T(0)$ is the base case). We can compute $T(1)=\max\{1,T(0)\}\geq 1$. We can then say that for $n>1$, $T(n)=nT(n-1)=n(n-1)T(n-2)=n(n-1)(n-2)T(n-3)=\cdots=n!T(0)=O(n!)$.
Many gave $O(n^n)$, and quite a few gave $O(n)$ as the number of times T would be called recursively (mistaking T for the program rather than the function).

iii) $T(n)=5T(n/25)+\sqrt{n}+\sqrt[3]{n}$. [2]

Answer:
$a=5,b=25,d=0.5$ so $\log_b a=\log_{25}5=0.5=d$ so:
$T(n)=O(n^d\log n)=O(\sqrt{n}\log n)$.
Most OK, some thought $\sqrt[3]{n}=n^{3/2}$.

iv) $T(n)=32T(n/2)+O(n^6)$. [2]

Answer:
$a=32,b=2,d=6$ so $\log_b a=\log_2 32=5<d$ so:
$T(n)=O(n^d)=O(n^6)$.
Almost all correct.

v) $T(n)=(16T(3n/4)+n^{1.5})/9$. [2]

Answer:

Answers

$a = 16/9, b = 4/3, d = 1.5$ *so* $\log_b a = \log_{4/3} 16/9 = 2 > d$ *so:*
$T(n) = O(n^{\log_b a}) = O(n^2)$.
Mostly OK, a few people thought $b = 3/4$ *or missed the 9. A couple thought the master theorem didn't apply.*

Answers

2. The merge sort algorithm is a divide and conquer algorithm for sorting an array of n items. It works by:

- **Dividing** the array into two sub-arrays of approximately equal size.
- **Sorting** the sub-arrays recursively.
- **Combining** the sorted sub-arrays into a single sorted array by jointly iterating over them.

The following pseudocode implements this algorithm:

```
1:  function MERGESORT(x)
2:      n = len(x)
3:      if n ≤ 1 then
4:          return x
5:      end if
6:      m = floor(n/2)
7:      L = MERGESORT(subarray of items 0 to m − 1 of x)
8:      R = MERGESORT(subarray of items m to n − 1 of x)
9:      y = array of size n
10:     i = 0, j = 0
11:     for k = 0 to n − 1 do
12:         r = False
13:         if i = m then
14:             r = True
15:         else if j < n − m then
16:             r = (R_j ≤ L_i)
17:         end if
18:         if r then
19:             y_k = R_j
20:             j = j + 1
21:         else
22:             y_k = L_i
23:             i = i + 1
24:         end if
25:     end for
26:     return y
27: end function
```

a) Explain the merging step (lines 11-25). [3]

Answer:

At this point in the algorithm we are jointly iterating over the two subarrays L and R, using indices i into L and j into R, and copying into the array y (indexed by k). We should copy over the next item R_j from subarray R, and increment its index j (lines 19-20), in one of two conditions: either (a) there are no items left in L, i.e. when $i = m$ (line 13), or (b) there are items left in both L and R (line 15), and item R_j is smaller than item L_i (line 16). If neither of these conditions holds (when r is False), we should copy over the next item from the L array and increment its index i (lines 22-23).

Most were able to do this well. A few marks were lost for not mentioning the case where all items from L had already been taken.

b) Calculate a tight upper bound on the complexity of the algorithm. [3]

Answer:

We calculate an upper bound using the Master Theorem. We divide into two

Answers

equal cases, and the combination operation is $O(n)$ therefore $T(n) = 2T(n/2) + O(n)$. Applying Master Theorem with $a = b = 2, d = 1$ gives $\log_b a = \log_2 2 = 1 = d$ so $T(n) = O(n \log n)$.
Most OK.

c) The *ordering* of a sorting algorithm's output is the output order of items from the input array. Assume each item is unique (no repeated items). Every possible ordering (or permutation) of n items must be a possible outcome of a sorting algorithm.

i) The array $\{A, B, C\}$ has 6 possible orderings after sorting: *ABC*, *ACB*, *BAC*, *BCA*, *CAB*, *CBA*. What is the number of possible outcomes when sorting an array of n unique items? [2]
Answer:
The number of permutations of n items is n!.
Almost all OK.

ii) If we know the result of a single comparison between two items in an array, this eliminates some possible orderings. Given knowledge of any single comparison, how many remaining outcomes are there for the sorted array? [2]
Answer:
There are two possible results of the comparison between two items, and each rules out half of the possible outcomes, so there are $n!/2$ remaining outcomes.
Quite a few gave $(n-1)!$ which is the number of permutations if you take one out, or $(n-2)!$ for taking two out.

iii) In the worst case, how many comparisons between items in an array would be required to fully sort the array? [3]
Answer:
Each comparison reduces the number of outcomes by a factor 2, so we need to solve $2^k = n!$, or $k = \text{ceil}(\log_2 n!)$.
Quite a few misunderstood this question and assumed a specific algorithm or input array, e.g. bubble sort on a reverse sorted array which gives $\Omega(n^2)$ steps.

iv) Using Ω notation, find a lower complexity bound for the number of operations for a sorting algorithm that can only use comparisons. Express your answer using only combinations of logarithmic, polynomial or exponential functions of n. [4]
Answer:
In the worst case we will need at least $\log_2 n!$ comparisons so at least this many operations will be required. So $\Omega(\log n!)$. We can simplify this by noting that $\log n! = \log \prod_{i=1}^{n} i = \sum_{i=1}^{n} \log i$; $\sum_{i=1}^{n} \log i \geq \sum_{i=n/2}^{n} \log i$; each term in the summation is $\log i \geq \log(n/2)$ since $i \geq n/2$; $\log n! \geq \sum_{i=n/2}^{n} \log(n/2) = (n/2)\log(n/2)$; and therefore $\Omega(\log n!) = \Omega(n \log n)$.
Similarly to above, many people assumed a particular algorithm on already sorted data to give $\Omega(n)$.

d) Consider an array x of $n = 2^m$ items. Consider the recursive depth d of each function call in the execution of the merge sort algorithm. So, the first function call executed is at depth $d = 1$ and this is the only function call executed at this depth. This first function call recursively calls merge sort twice (for the arrays L and R), and these two function calls will be at depth $d = 2$, and so on.

Answers

At depth d, each function call is required to combine two sorted arrays, each of size 2^{m-d}, into a single sorted array of size 2^{m-d+1}. There are 2^{d-1} such function calls executed.

Let C_d refer to the total number of comparisons $R_j \leq L_i$ made on line 16 across all function calls executed at depth d.

i) What is the minimum possible number of comparisons C_d at depth d?

Hint: first consider how many comparisons are made in a single function call at this depth. [4]

Answer:
For each pair of sub-arrays of size $a = 2^{m-d}$ we need to add $2a$ items to the output array. By looking at the pseudocode above, we can see that each append to the output array requires either one or zero comparisons. We want to minimise the number of comparisons, so we want the condition $i = a$ or $j = a$ to be true as often as possible. This happens when one of the two pointers i or j reaches the end of an array, because then we just append the remaining items from the other array. To get to this point most quickly requires a comparisons (each item of one array with the first item of the other array). So each pair requires at least $a = 2^{m-d}$ comparisons and there are 2^{d-1} pairs for a total of $C_d = n/2 = 2^{m-1}$ comparisons.
Quite a few forgot to multiply by 2^{d-1}, and a few got the indices off by 1 (e.g. using 2^d instead of 2^{d-1}).

ii) What is the maximum possible number of comparisons C_d at depth d? [4]

Answer:
As above we consider a pair of sub-arrays of size $a = 2^{m-d}$ being combined. There are $2a$ items to be processed at the start, and after each step one item no longer requires processing. When there is only one item remaining no comparison is necessary, but in all other cases one comparison might be needed. Therefore, at most $2a - 1 = 2^{m-d+1} - 1$ comparisons are needed for each pair. There are 2^{d-1} pairs for a total of $C_d = 2^m - 2^{d-1} = n(1 - 1/2^{m-d+1})$ comparisons.
Similar to previous part, except in addition many forgot the -1, giving 2^{m-d+1} as the number per function call.

iii) Give an exact upper and lower bound for the total number of comparisons $C = \sum_{d=1}^{m} C_d$ required by the merge sort algorithm for $n = 2^m$ items. [3]

Answer:
For the lower bound, we know that at each depth d there are $C_d \geq n/2$ comparisons required. There are $m = \log_2 n$ stages, and so the number of comparisons is $C \geq (n/2)\log_2 n$.

For the upper bound, we know that at each depth d there are $C_d \leq n(1 - 1/2^{m-d+1})$ comparisons required. So the upper bound on the number of comparisons is $\sum_{d=1}^{m} n(1 - 1/2^{m-d+1}) = n(m - \sum_{p=1}^{m} 1/2^p)$. We can simpify the geometric series as $\sum_{p=1}^{m} 1/2^p = (1/2 - 1/2^{m+1})/(1 - 1/2) = 1 - 1/2^m = (1 - 1/n)$. Putting that together, the maximum number of comparisons is $C \leq n(m - (1 - 1/n)) = nm - n + 1 = n\log_2 n - n + 1$.
Relatively few perfect answers, some difficulty with geometric series.

Answers

iv) Find an array x of $n = 4$ items that takes the maximum number of comparisons using merge sort, and an array of the same size that takes the minimum number of comparisons. [2]

Answer:

Using the formula above, the minimum number of comparisons is $(n/2)\log_2 n = 4$. *The maximum number is* $n\log_2 n - n + 1 = 5$.

If $x = \{1,2,3,4\}$ *then at depth 2, pair* $\{1\},\{2\}$ *requires a single comparison to combine to* $\{1,2\}$ *and pair* $\{3\},\{4\}$ *requires a single comparison to combine to* $\{3,4\}$. *At depth 1, pair* $\{1,2\},\{3,4\}$ *requires one comparison to insert the 1, one comparison to insert the 2, and no further comparisons. So a total of 4 comparisons.*

If $x = \{4,2,3,1\}$ *then at depth 2, pair* $\{4\},\{2\}$ *requires a single comparison to combine to* $\{2,4\}$, *and pair* $\{3\},\{1\}$ *requires a single comparison to combine to* $\{1,3\}$. *At depth 1, a single comparison is required to insert the 1 (compare 2 and 1); a second to insert the 2 (compare 2 and 3); and a third to insert the 3 (compare 3 and 4). So a total of 5 comparisons.*

Most found examples but didn't give all the reasoning.

Master Theorem. If $T(n)$ satisfies

$$T(n) = a\,T(\,n/b\,) \;+\; O(n^d)$$

for some $a > 0$, $b > 1$ and $d \geq 0$, then

$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \\ O(n^d \log n) & \text{if } d = \log_b a \\ O(n^{\log_b a}) & \text{if } d < \log_b a \end{cases}$$

Answers